

PROJECT CONCEPT REPORT
REFURBISHED LAPTOP DIAGNOSTIC TOOL - WINDOWS EXE
October 2025

This project involves developing a portable Windows executable diagnostic tool that runs on pre-installed Windows operating systems to perform comprehensive hardware analysis. Built as a standalone application with no installation required, the system will detect and test all major components including CPU, memory, storage devices, and peripherals, then generate professional diagnostic reports.

1. System Overview

A portable Windows executable that:

- Runs on any Windows 10/11 system without installation
- Performs comprehensive hardware detection and testing
- Generates detailed diagnostic reports
- Stores results to USB drive or uploads to cloud/network
- Requires administrator privileges for hardware access

2. Technical Architecture

2.1 Technology Stack

Layer	Technology	Justification
Execution Environment	C# .NET 8 or C++	Native Windows API access, single-file deployment, excellent performance
UI Framework	WPF (Windows Presentation Foundation)	Modern interface, hardware-accelerated, native Windows integration
Hardware Access	WMI (Windows Management Instrumentation), Windows APIs, Kernel-mode drivers (optional)	Industry standard for Windows hardware queries, comprehensive coverage
Diagnostic Libraries	LibreHardwareMonitor, CrystalDiskInfo Library, HWiNFO SDK (optional commercial)	Production-ready, actively maintained, proven accuracy
Database	SQLite (embedded)	Portable, no server required, lightweight, reliable

Report Format HTML/PDF

Portable, professional, easy to share and verify

2.2 Deployment Package

- Single-file executable (50-100MB approx with embedded runtime)
- Administrator privilege auto-elevation
- Embedded SQLite database for local storage
- All dependencies bundled

3. Diagnostic Capabilities

3.1 Hardware Detection & Testing

Component	Detection Method	Testing Capability	Maturity
Processor	WMI (Win32_Processor), CPUID instruction	Stress testing, thermal monitoring, frequency validation, core detection	Excellent
Memory (RAM)	WMI (Win32_PhysicalMemory), Windows Memory Diagnostic API	Pattern testing, allocation tests, bandwidth benchmarks	Good (limited vs memtest86+)
Storage Devices	WMI (Win32_DiskDrive), SMART APIs, NVMe passthrough	SMART health, bad block detection, performance benchmarks, wear leveling	Excellent
Motherboard	WMI (Win32_BaseBoard), SMBIOS	Model, BIOS version, chipset information, serial numbers	Excellent
Graphics Card	WMI (Win32_VideoController), DirectX API, OpenGL	Detection, VRAM size, driver version, 3D stress testing, artifact detection	Excellent

Network Adapters	WMI (Win32_NetworkAdapter), netsh	Chipset, driver version, link speed, MAC address, connectivity test	Excellent
USB Devices	WMI (Win32_USBController), Device Manager API	Full enumeration, device tree, USB version detection	Excellent
Temperature Sensors	LibreHardwareMonitor, OpenHardwareMonitor libraries	Real-time monitoring (CPU, GPU, storage, motherboard)	Good (hardware-dependent)
Battery (Laptops)	WMI (Win32_Battery), Battery Report API	Health, capacity, cycle count, charge/discharge rate, wear level	Excellent
Audio/Peripherals	WMI, DirectSound, Core Audio API	Detection, playback test, recording test	Excellent
Display	WMI (Win32_DesktopMonitor), EDID parsing	Resolution, refresh rate, panel type, connection type	Excellent
Webcam	DirectShow, Windows Media Foundation	Detection, resolution, test capture	Excellent
Keyboard/Touchpad	Raw Input API, WMI	Key detection, dead key identification, touchpad functionality	Good

3.2 Diagnostic Tool Suite

Core Windows APIs & Methods:

- WMI (Windows Management Instrumentation): Primary hardware detection and management
- SMART API: Storage health monitoring and diagnostics
- Windows Performance Counters: Real-time system metrics
- Device Manager API: Hardware enumeration and status
- DirectX/OpenGL: GPU testing and validation
- Windows Memory Diagnostic API: Memory testing integration

Third-Party Open-Source Libraries:

- LibreHardwareMonitor (MPL 2.0): Temperature, fan speed, voltage, clock monitoring
- CrystalDiskInfo Library (MIT): SMART parsing, storage health, SSD metrics

- MemTest Windows adaptation: Windows-based memory pattern testing
- Custom stress testing: CPU/GPU load generation

Total Timeline: 12 weeks (3 months) for production-ready v1.0 (THIS IS ASSUMING 1-2 DEVELOPERS WORKING FULL-TIME)